

PRACTICAL WEEK-4

NAME:CH.SAI VIHAMSH

ROLL:2520090090

SESSION-4

Task 1: Parent creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behaviour of both the functions.

```
GNU nano 8.7.1 wait_waitpid.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    printf("Parent process started. PID = %d\n", getpid());

    for (int i = 1; i <= 3; i++)
    {
        pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (pid == 0)
        {
            printf("Child %d: PID = %d, Parent PID = %d\n",
                   i, getpid(), getppid());

            sleep(i);

            printf("Child %d completed.\n", i);
            exit(i);
        }
    }

    printf("\nParent is waiting using wait()...\n");

    for (int i = 0; i < 3; i++)
```

```

        if (pid > 0)
        {
            printf("wait(): Child PID %d terminated with status %d\n",
                pid, WEXITSTATUS(status));
        }
    }

    printf("\nAll children completed using wait().\n");
    printf("\nCreating one more child for waitpid()...\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        printf("waitpid child: PID = %d\n", getpid());
        sleep(2);
        printf("waitpid child completed.\n");
        exit(10);
    }

    printf("Parent waiting specifically for PID %d using waitpid()...\n", p
waitpid(pid, &status, 0);

    printf("waitpid(): Child PID %d terminated with status %d\n",
        pid, WEXITSTATUS(status));

    printf("Parent process completed.\n");

```

```

saiviamsh@Varshini-PC:~$ nano pweek4.c
saiviamsh@Varshini-PC:~$ gcc pweek4.c -o pweek4
saiviamsh@Varshini-PC:~$ ./pweek4
Parent process started. PID = 545
Child 1: PID = 546, Parent PID = 545
Child 2: PID = 547, Parent PID = 545

Parent is waiting using wait()...
Child 3: PID = 548, Parent PID = 545
Child 1 completed.
wait(): Child PID 548 terminated with status 1
Child 2 completed.
wait(): Child PID 548 terminated with status 2
Child 3 completed.
wait(): Child PID 548 terminated with status 3

All children completed using wait().

Creating one more child for waitpid()...
Parent waiting specifically for PID 549 using waitpid().
waitpid child: PID = 549
waitpid child completed.
waitpid(): Child PID 549 terminated with status 10
Parent process completed.
saiviamsh@Varshini-PC:~$ |

```

Observation: The program successfully created three child processes using fork(). The parent process used wait() to wait for the termination of its child processes and collected their exit statuses successfully. The program then created another child and used waitpid() to wait specifically for that child's PID. This demonstrates that wait() can wait for any terminated child, whereas waitpid() provides control to wait for a specific child process.

Result: The parent process successfully synchronized with all child processes using both wait() and waitpid(), and no child processes were left uncollected.

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process is terminating...\n");
        exit(0);
    }
    else
    {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child process PID = %d\n", pid);
        printf("Parent is sleeping for 30 seconds...\n");
        printf("During this time, the child becomes a zombie.\n");

        sleep(30);

        printf("Parent process completed.\n");
    }

    return 0;
}
```

```
saiviamsh@Varshini-PC:~$ nano pweek4t2.c
saiviamsh@Varshini-PC:~$ gcc pweek4t2.c -o pweek4t2
saiviamsh@Varshini-PC:~$ ./pweek4t2
Child process: PID = 577
Child process is terminating...
Parent process: PID = 576
Child process PID = 577
Parent is sleeping for 30 seconds...
During this time, the child becomes a zombie.
Parent process completed.
saiviamsh@Varshini-PC:~$ |
```

Observation: The program successfully created a parent and child process using `fork()`. The child process terminated while the parent process continued executing and slept for 30 seconds. Since the parent did not call `wait()` to collect the child's termination status, the terminated child temporarily remained in the process table as a zombie process (Z state). After the parent process completed, the zombie was removed.

The program was then modified to use proper synchronization with `wait()`, allowing the parent to collect the child's termination status and preventing the creation of a zombie process.

Result: A zombie process was successfully created and observed, and proper synchronization using `wait()` was used to eliminate it.